Modular Arithmetic and GCDs

COMP2711 Discrete Mathematical Tools

Mingxun Zhou (mingxunz@ust.hk)

Contents

1. Introduction

2. Congruence

3. Modular computation and reasoning

4. GCDs and the Euclidean algorithm

5. Algorithms and recap

What is number theory?

Number theory studies the properties and relationships of integers.

$$\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots$$

What is number theory?

Number theory studies the properties and relationships of integers.

$$\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots$$

Factors and primes

$$60 = 2^2 \cdot 3 \cdot 5$$

Primes are the multiplicative building blocks of integers greater than 1.

What is number theory?

Number theory studies the properties and relationships of integers.

$$\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots$$

Factors and primes

$$60 = 2^2 \cdot 3 \cdot 5$$

Primes are the multiplicative building blocks of integers greater than 1.

Integer solutions

$$3^2 + 4^2 = 5^2$$

Which equations have integer solutions?

Simple questions, deep problems

Goldbach's conjecture: Can every even integer greater than 2 be written as the sum of two primes?

Simple questions, deep problems

Goldbach's conjecture: Can every even integer greater than 2 be written as the sum of two primes?

$$8 = 3 + 5, \qquad 20 = 3 + 17.$$

Simple questions, deep problems

Goldbach's conjecture: Can every even integer greater than 2 be written as the sum of two primes?

$$8 = 3 + 5, \qquad 20 = 3 + 17.$$

Your turn: can you find two primes that add to 100?

$$100 = ? + ?$$

Simple questions, deep problems

Goldbach's conjecture: Can every even integer greater than 2 be written as the sum of two primes?

$$8 = 3 + 5, \qquad 20 = 3 + 17.$$

Your turn: can you find two primes that add to 100?

$$100 = ? + ?$$

Goldbach's conjecture dates to letters between Goldbach and Euler in **1742**.

Simple questions, deep problems

Goldbach's conjecture: Can every even integer greater than 2 be written as the sum of two primes?

$$8 = 3 + 5, \qquad 20 = 3 + 17.$$

Your turn: can you find two primes that add to 100?

$$100 = ? + ?$$

Goldbach's conjecture dates to letters between Goldbach and Euler in **1742**.

Computers have verified every even integer $4 \leq n \leq 4 \times 10^{18}$.

But the general conjecture remains open — **284 years later** (2026).

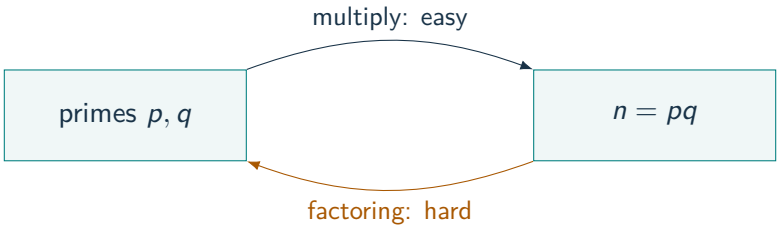
Why does computer science care?

Not just finding answers: **how efficiently can we compute them?**

Why does computer science care?

Not just finding answers: **how efficiently can we compute them?**

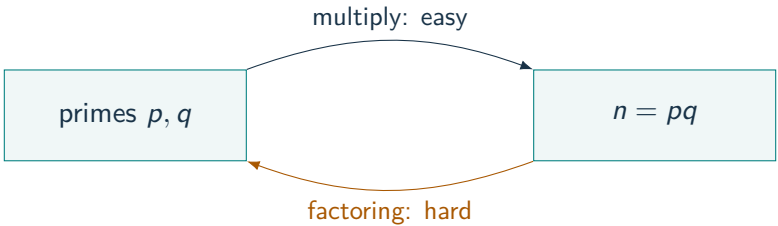
Cryptography: some integer computations are easy to perform but difficult to reverse.



Why does computer science care?

Not just finding answers: **how efficiently can we compute them?**

Cryptography: some integer computations are easy to perform but difficult to reverse.



The asymmetry allows us to build cryptography!

Notation review: Number sets

Throughout this lecture series, all numbers are assumed to be integers unless stated otherwise.

Notation review: Number sets

Throughout this lecture series, all numbers are assumed to be integers unless stated otherwise.

Natural numbers	$\mathbb{N} = \{0, 1, 2, 3, \ldots\}$
All integers	$\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$
Positive integers	$\mathbb{N}^+ = \mathbb{Z}^+ = \{1, 2, 3, \ldots\}$

Review: Integer division

Over the real numbers, $17/3 = 5.666666\ldots$.

Review: Integer division

Over the real numbers, $17/3 = 5.666666\ldots$.

In integer division, we keep an integer quotient and a remainder:

$$17 = 3 \times \underbrace{5}_{\text{quotient}} + \underbrace{2}_{\text{remainder}} .$$

Review: Integer division

Over the real numbers, $17/3 = 5.666666\ldots$.

In integer division, we keep an integer quotient and a remainder:

$$17 = 3 \times \underbrace{5}_{\text{quotient}} + \underbrace{2}_{\text{remainder}} .$$



five groups of three

two left over

$$17 \operatorname{div} 3 = 5, \qquad 17 \bmod 3 = 2.$$

Notation review: Quotient and remainder

For every a and $d > 0$, there exist unique q, r with

$$\underbrace{a}_{\text{dividend}} = \underbrace{d}_{\text{divisor}} \underbrace{q}_{\text{quotient}} + \underbrace{r}_{\text{remainder}}, \qquad 0 \leq r < d.$$

Notation review: Quotient and remainder

For every a and $d > 0$, there exist unique q, r with

$$\underbrace{a}_{\text{dividend}} = \underbrace{d}_{\text{divisor}} \underbrace{q}_{\text{quotient}} + \underbrace{r}_{\text{remainder}}, \qquad 0 \leq r < d.$$

$$q = a \operatorname{div} d, \qquad r = a \bmod d.$$

Notation review: Quotient and remainder

For every a and $d > 0$, there exist unique q, r with

$$\underbrace{a}_{\text{dividend}} = \underbrace{d}_{\text{divisor}} \underbrace{q}_{\text{quotient}} + \underbrace{r}_{\text{remainder}}, \qquad 0 \le r < d.$$

$$q = a \operatorname{div} d, \qquad r = a \bmod d.$$

$$17 = 3 \cdot 5 + 2$$

$$17 \operatorname{div} 3 = 5, \quad 17 \bmod 3 = 2$$

$$-17 = 3(-6) + 1$$

$$-17 \operatorname{div} 3 = -6, \quad -17 \bmod 3 = 1$$

Divisibility: Zero remainder

For $a > 0$,

$$\boxed{a \mid b \iff b \bmod a = 0 \iff b = ak \text{ for some } k.}$$

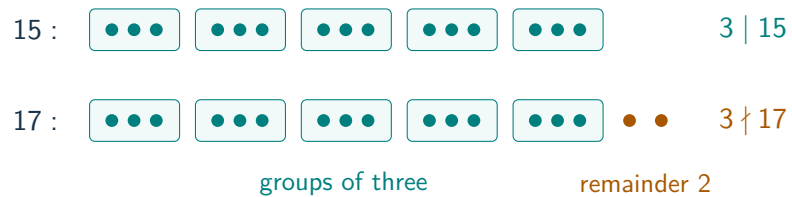
Read: a **divides** b.

Divisibility: Zero remainder

For $a > 0$,

$$a \mid b \iff b \bmod a = 0 \iff b = ak \text{ for some } k.$$

Read: *a* **divides** *b*.



Contents

1. Introduction

2. Congruence

3. Modular computation and reasoning

4. GCDs and the Euclidean algorithm

5. Algorithms and recap

Congruence

MUST UNDERSTAND Congruence

For $m > 0$,

$$a \equiv b \pmod{m} \iff a \bmod m = b \bmod m \iff m \mid (a - b).$$

To read it out: a is congruent to b modulo m.

Congruence

MUST UNDERSTAND Congruence

For $m > 0$,

$$a \equiv b \pmod{m} \iff a \bmod m = b \bmod m \iff m \mid (a - b).$$

To read it out: a is congruent to b modulo m.

$$17 \equiv 2 \pmod{5} \iff 17 \bmod 5 = 2 \bmod 5 \iff 5 \mid (17 - 2)$$

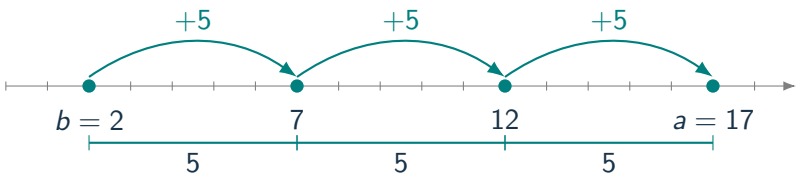
$$2 = 2 \qquad 17 - 2 = 15 = 5 \cdot 3$$

Congruence: Steps on a number line

$a \equiv b \pmod{m}$ means $a = km + b$ for some integer k.

Congruence: Steps on a number line

$a \equiv b \pmod{m}$ means $a = km + b$ for some integer k.



Arithmetic with congruences

MUST UNDERSTAND Arithmetic compatibility

Let $m > 0$. Suppose $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$.

$$a + c \equiv b + d \pmod{m},$$

Arithmetic with congruences

MUST UNDERSTAND Arithmetic compatibility

Let $m > 0$. Suppose $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$.

$$a + c \equiv b + d \pmod{m},$$
$$a - c \equiv b - d \pmod{m},$$

Arithmetic with congruences

MUST UNDERSTAND Arithmetic compatibility

Let $m > 0$. Suppose $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$.

$$\begin{aligned} a + c &\equiv b + d \pmod{m}, \\ a - c &\equiv b - d \pmod{m}, \\ ac &\equiv bd \pmod{m}. \end{aligned}$$

Arithmetic with congruences

MUST UNDERSTAND Arithmetic compatibility

Let $m > 0$. Suppose $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$.

$$a + c \equiv b + d \pmod{m},$$
$$a - c \equiv b - d \pmod{m},$$
$$ac \equiv bd \pmod{m}.$$

For any fixed positive integer n, repeated multiplication gives

$$a^n \equiv b^n \pmod{m}.$$

Why multiplication preserves congruence

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

Why multiplication preserves congruence

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

In general, to prove $X \equiv Y \pmod{m}$, consider their difference:

$$X \equiv Y \pmod{m} \iff X - Y \equiv 0 \pmod{m}.$$

Why multiplication preserves congruence

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

In general, to prove $X \equiv Y \pmod{m}$, consider their difference:

$$X \equiv Y \pmod{m} \iff X - Y \equiv 0 \pmod{m}.$$

Find an integer k such that

$$X - Y = k \cdot m.$$

Why multiplication preserves congruence

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

In general, to prove $X \equiv Y \pmod{m}$, consider their difference:

$$X \equiv Y \pmod{m} \iff X - Y \equiv 0 \pmod{m}.$$

Find an integer k such that

$$X - Y = k \cdot m.$$

That is, X is shifted from Y by k steps of size m.

Why multiplication preserves congruence: Proof

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

It is sufficient to prove that $ac - bd$ is an integer multiple of m.

Why multiplication preserves congruence: Proof

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

It is sufficient to prove that $ac - bd$ is an integer multiple of m.

By definition of congruence, we can write

$$a = km + b, \qquad c = \ell m + d \qquad \text{for integers } k, \ell.$$

Why multiplication preserves congruence: Proof

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

It is sufficient to prove that $ac - bd$ is an integer multiple of m.

By definition of congruence, we can write

$$a = km + b, \qquad c = \ell m + d \qquad \text{for integers } k, \ell.$$

$$ac - bd = (km + b)(\ell m + d) - bd$$

Why multiplication preserves congruence: Proof

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

It is sufficient to prove that $ac - bd$ is an integer multiple of m.

By definition of congruence, we can write

$$a = km + b, \qquad c = \ell m + d \qquad \text{for integers } k, \ell.$$

$$\begin{aligned} ac - bd &= (km + b)(\ell m + d) - bd \\ &= k\ell m^2 + kmd + b\ell m \end{aligned}$$

Why multiplication preserves congruence: Proof

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

It is sufficient to prove that $ac - bd$ is an integer multiple of m.

By definition of congruence, we can write

$$a = km + b, \qquad c = \ell m + d \qquad \text{for integers } k, \ell.$$

$$\begin{aligned} ac - bd &= (km + b)(\ell m + d) - bd \\ &= k\ell m^2 + kmd + b\ell m \\ &= \underbrace{(k\ell m + kd + b\ell)}_{\text{an integer}} \cdot m. \end{aligned}$$

Why multiplication preserves congruence: Proof

Assume $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, where $m > 0$.

To prove: $ac \equiv bd \pmod{m}$.

It is sufficient to prove that $ac - bd$ is an integer multiple of m.

By definition of congruence, we can write

$$a = km + b, \qquad c = \ell m + d \qquad \text{for integers } k, \ell.$$

$$\begin{aligned} ac - bd &= (km + b)(\ell m + d) - bd \\ &= k\ell m^2 + kmd + b\ell m \\ &= \underbrace{(k\ell m + kd + b\ell)}_{\text{an integer}} \cdot m. \end{aligned}$$

Thus ac differs from bd by a multiple of m: $ac \equiv bd \pmod{m}$.

Practice: Substitute a remainder

Suppose $n \equiv 2 \pmod 5$. What is the remainder of $n^2 + n$?

Practice: Substitute a remainder

Suppose $n \equiv 2 \pmod 5$. What is the remainder of $n^2 + n$?

$$n^2 + n \equiv 2^2 + 2 = 6 \equiv \boxed{1} \pmod 5.$$

Contents

1. Introduction

2. Congruence

3. Modular computation and reasoning

4. GCDs and the Euclidean algorithm

5. Algorithms and recap

Reduce before calculating

Compute $(38 \cdot 47 + 25 \cdot 19) \bmod 12$.

$38 \qquad 47 \qquad 25 \qquad 19$

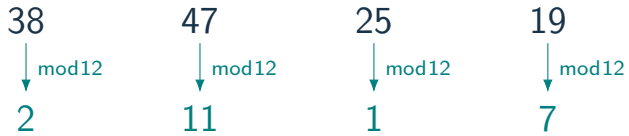
Reduce before calculating

Compute $(38 \cdot 47 + 25 \cdot 19) \bmod 12$.



Reduce before calculating

Compute $(38 \cdot 47 + 25 \cdot 19) \bmod 12$.



$$38 \cdot 47 + 25 \cdot 19 \equiv 2 \cdot 11 + 1 \cdot 7 = 29 \equiv \boxed{5} \pmod{12}.$$

Warning: Cannot reduce the exponent trivially!

Can we reduce the exponent modulo m?

$$a^b \stackrel{?}{\equiv} a^{b \bmod m} \pmod{m}$$

Warning: Cannot reduce the exponent trivially!

Can we reduce the exponent modulo m?

$$a^b \stackrel{?}{\equiv} a^{b \bmod m} \pmod{m}$$

No!! **Counterexample:** $a = 2$, $b = 6$, $m = 5$, so $6 \bmod 5 = 1$.

$$2^6 = 64 \equiv 4 \pmod{5}, \qquad 2^{6 \bmod 5} = 2^1 = 2.$$

Warning: Cannot reduce the exponent trivially!

Can we reduce the exponent modulo m?

$$a^b \stackrel{?}{\equiv} a^{b \bmod m} \pmod{m}$$

No!! **Counterexample:** $a = 2$, $b = 6$, $m = 5$, so $6 \bmod 5 = 1$.

$$2^6 = 64 \equiv 4 \pmod{5}, \qquad 2^{6 \bmod 5} = 2^1 = 2.$$

Valid rule (next lecture):

$$a^b \equiv a^{b \bmod \varphi(m)} \pmod{m},$$

where $m > 1$, $b \geq 0$, $\gcd(a, m) = 1$, and $\varphi(m)$ is Euler's totient function.

Modular powers: Two solutions

Compute $2^{13} \bmod 5$.

Modular powers: Two solutions

Compute $2^{13} \bmod 5$.

MUST UNDERSTAND Two required methods

Solution 1: Cycle of powers
Find a repeating cycle of remainders, then use its length.

Solution 2: Repeated squaring (fast exponentiation)
Build powers $a, a^2, a^4, a^8, \ldots$, then multiply the selected powers.

Solution 1: Build the cycle

Compute $2^{13} \bmod 5$. Start with $2^1 \bmod 5 = 2$.

Multiply the previous remainder by 2:

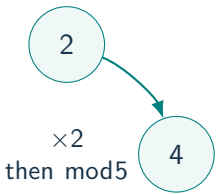


Solution 1: Build the cycle

Compute $2^{13} \bmod 5$. Start with $2^1 \bmod 5 = 2$.

Multiply the previous remainder by 2:

$$2^2 = 2 \cdot 2 \equiv 4 \pmod 5,$$



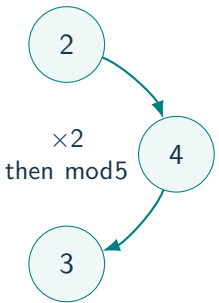
Solution 1: Build the cycle

Compute $2^{13} \bmod 5$. Start with $2^1 \bmod 5 = 2$.

Multiply the previous remainder by 2:

$$2^2 = 2 \cdot 2 \equiv 4 \pmod 5,$$

$$2^3 \equiv 4 \cdot 2 = 8 \equiv 3 \pmod 5,$$



Solution 1: Build the cycle

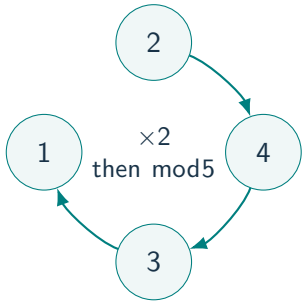
Compute $2^{13} \bmod 5$. Start with $2^1 \bmod 5 = 2$.

Multiply the previous remainder by 2:

$$2^2 = 2 \cdot 2 \equiv 4 \pmod 5,$$

$$2^3 \equiv 4 \cdot 2 = 8 \equiv 3 \pmod 5,$$

$$2^4 \equiv 3 \cdot 2 = 6 \equiv 1 \pmod 5,$$



Solution 1: Build the cycle

Compute $2^{13} \bmod 5$. Start with $2^1 \bmod 5 = 2$.

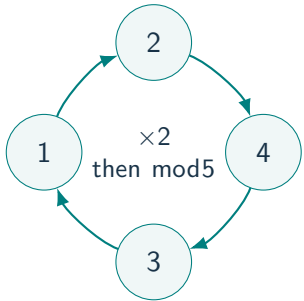
Multiply the previous remainder by 2:

$$2^2 = 2 \cdot 2 \equiv 4 \pmod 5,$$

$$2^3 \equiv 4 \cdot 2 = 8 \equiv 3 \pmod 5,$$

$$2^4 \equiv 3 \cdot 2 = 6 \equiv 1 \pmod 5,$$

$$2^5 \equiv 1 \cdot 2 = 2 \pmod 5.$$



Solution 1: Use the cycle

Compute 2^{13} mod 5.

Solution 1: Use the cycle

Compute $2^{13} \bmod 5$.

$$2^4 \equiv 1 \pmod 5 \quad \Longrightarrow \quad 2^{n+4} = 2^n 2^4 \equiv 2^n \pmod 5 \quad (n \geq 0).$$

The remainders repeat every 4 powers.

Solution 1: Use the cycle

Compute $2^{13} \bmod 5$.

$$2^4 \equiv 1 \pmod 5 \quad \Longrightarrow \quad 2^{n+4} = 2^n 2^4 \equiv 2^n \pmod 5 \quad (n \geq 0).$$

The remainders repeat every 4 powers.

$$13 = 3 \cdot 4 + 1$$

$$2^{13} = (2^4)^3 \cdot 2 \equiv 1^3 \cdot 2 = \boxed{2} \pmod 5.$$

Solution 2: Fast exponentiation

Compute 2^{13} mod 5.

Power	Remainder modulo 5	Square the previous remainder
2^1	2	starting value

Solution 2: Fast exponentiation

Compute 2^{13} mod 5.

Power	Remainder modulo 5	Square the previous remainder
2^1	2	starting value
2^2	4	$2^2 = 4$

Solution 2: Fast exponentiation

Compute 2^{13} mod 5.

Power	Remainder modulo 5	Square the previous remainder
2^1	2	starting value
2^2	4	$2^2 = 4$
2^4	1	$4^2 = 16 \equiv 1 \pmod 5$

Solution 2: Fast exponentiation

Compute 2^{13} mod 5.

Power	Remainder modulo 5	Square the previous remainder
2^1	2	starting value
2^2	4	$2^2 = 4$
2^4	1	$4^2 = 16 \equiv 1 \pmod 5$
2^8	1	$1^2 = 1$

Which of these powers make 2^{13}?

Solution 2: Binary decomposition

$$13 = (1101)_2 = 8 + 4 + 1 \qquad \Longrightarrow \qquad 2^{13} = 2^8 \cdot 2^4 \cdot 2.$$

Solution 2: Binary decomposition

$$13 = (1101)_2 = 8 + 4 + 1 \qquad \Longrightarrow \qquad 2^{13} = 2^8 \cdot 2^4 \cdot 2.$$

2^1	2^2	2^4	2^8
2	4	1	1

remainders modulo 5

Solution 2: Binary decomposition

$$13 = (1101)_2 = 8 + 4 + 1 \qquad \Longrightarrow \qquad 2^{13} = 2^8 \cdot 2^4 \cdot 2.$$

2^1	2^2	2^4	2^8
2	4	1	1

remainders modulo 5

$$2^{13} \equiv 1 \cdot 1 \cdot 2 = \boxed{2} \pmod 5.$$

Further practice: Two methods

Compute

$$3^{14} \bmod 10.$$

Further practice: Two methods

Compute

$$3^{14} \bmod 10.$$

Use both methods:

1. Find a cycle of powers.
2. Use fast exponentiation (repeated squaring).

Further practice: Solution 1

Compute 3^{14} mod 10 using a cycle of powers.

Further practice: Solution 1

Compute 3^{14} mod 10 using a cycle of powers.

n	1	2	3	4	5
3^n mod 10	3	9	7	1	3

Further practice: Solution 1

Compute $3^{14} \bmod 10$ using a cycle of powers.

n	1	2	3	4	5
$3^n \bmod 10$	3	9	7	1	3

Since $3^4 \equiv 1 \pmod{10}$, the remainders repeat every 4 powers.

$$14 = 3 \cdot 4 + 2$$

$$3^{14} = (3^4)^3 \cdot 3^2 \equiv 1^3 \cdot 9 = \boxed{9} \pmod{10}.$$

Further practice: Solution 2

Compute 3^{14} mod 10 using fast exponentiation.

Further practice: Solution 2

Compute 3^{14} mod 10 using fast exponentiation.

Power	Remainder modulo 10	Computation
3^1	3	starting value
3^2	9	$3^2 = 9$
3^4	1	$9^2 = 81 \equiv 1 \pmod{10}$
3^8	1	$1^2 = 1$

Further practice: Solution 2

Compute 3^{14} mod 10 using fast exponentiation.

Power	Remainder modulo 10	Computation
3^1	3	starting value
3^2	9	$3^2 = 9$
3^4	1	$9^2 = 81 \equiv 1 \pmod{10}$
3^8	1	$1^2 = 1$

Binary decomposition: $14 = (1110)_2 = 8 + 4 + 2$.

Further practice: Solution 2

Compute 3^{14} mod 10 using fast exponentiation.

Power	Remainder modulo 10	Computation
3^1	3	starting value
3^2	9	$3^2 = 9$
3^4	1	$9^2 = 81 \equiv 1 \pmod{10}$
3^8	1	$1^2 = 1$

Binary decomposition: $14 = (1110)_2 = 8 + 4 + 2$.

$$3^{14} = 3^8 \cdot 3^4 \cdot 3^2 \equiv 1 \cdot 1 \cdot 9 = \boxed{9} \pmod{10}.$$

Why cancellation can fail

The operations $+$, $-$, and $\times$ preserve congruence.

For $m > 0$, if $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then

$$a \pm c \equiv b \pm d \pmod{m}, \qquad ac \equiv bd \pmod{m}.$$

How about $\div$?

Why cancellation can fail

The operations $+$, $-$, and $\times$ preserve congruence.

For $m > 0$, if $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then

$$a \pm c \equiv b \pm d \pmod{m}, \qquad ac \equiv bd \pmod{m}.$$

How about $\div$?

No! Not in general.

Counterexample: divide both sides by 2.

$$2 \equiv 8 \pmod{6}, \qquad \text{but} \qquad \frac{2}{2} = 1 \not\equiv 4 = \frac{8}{2} \pmod{6}.$$

Contents

1. Introduction

2. Congruence

3. Modular computation and reasoning

4. GCDs and the Euclidean algorithm

5. Algorithms and recap

Divisor and prime numbers

Divisor: if $a \mid b$, we say a is a divisor of b.

For example, $3 \mid 15$, so 3 is a divisor of 15.

Divisor and prime numbers

Divisor: if $a \mid b$, we say a is a divisor of b.

For example, $3 \mid 15$, so 3 is a divisor of 15.

A **non-trivial divisor** of $n > 1$ is a positive divisor other than 1 and n.

For example, 3 and 5 are the non-trivial divisors of 15.

Divisor and prime numbers

Divisor: if $a \mid b$, we say a is a divisor of b.

For example, $3 \mid 15$, so 3 is a divisor of 15.

A **non-trivial divisor** of $n > 1$ is a positive divisor other than 1 and n.

For example, 3 and 5 are the non-trivial divisors of 15.

An integer $p > 1$ is **prime** if its only positive divisors are 1 and p.

$$2,\ 3,\ 5,\ 7,\ 11, \ldots$$

An integer greater than 1 that is not prime is **composite**.

Divisor and prime numbers

Divisor: if $a \mid b$, we say a is a divisor of b.

For example, $3 \mid 15$, so 3 is a divisor of 15.

A **non-trivial divisor** of $n > 1$ is a positive divisor other than 1 and n.

For example, 3 and 5 are the non-trivial divisors of 15.

An integer $p > 1$ is **prime** if its only positive divisors are 1 and p.

$$2,\ 3,\ 5,\ 7,\ 11, \ldots$$

An integer greater than 1 that is not prime is **composite**.

Is 0 or 1 prime or composite?

Divisor and prime numbers

Divisor: if $a \mid b$, we say a is a divisor of b.

For example, $3 \mid 15$, so 3 is a divisor of 15.

A **non-trivial divisor** of $n > 1$ is a positive divisor other than 1 and n.

For example, 3 and 5 are the non-trivial divisors of 15.

An integer $p > 1$ is **prime** if its only positive divisors are 1 and p.

$$2,\ 3,\ 5,\ 7,\ 11, \ldots$$

An integer greater than 1 that is not prime is **composite**.

Is 0 or 1 prime or composite?

Neither. Both definitions require an integer greater than 1.

Fundamental theorem of arithmetic

Factorization means writing a number as a product of factors.

$$60 = 6 \times 10.$$

Fundamental theorem of arithmetic

Factorization means writing a number as a product of factors.

$$60 = 6 \times 10.$$

In a **prime factorization**, every factor is prime.

$$60 = 6 \times 10 = (2 \times 3) \times (2 \times 5) = 2^2 \times 3 \times 5.$$

Fundamental theorem of arithmetic

Factorization means writing a number as a product of factors.

$$60 = 6 \times 10.$$

In a **prime factorization**, every factor is prime.

$$60 = 6 \times 10 = (2 \times 3) \times (2 \times 5) = 2^2 \times 3 \times 5.$$

A prime cannot be split into smaller positive integer factors: its only positive divisors are 1 and itself.

Fundamental theorem of arithmetic

Factorization means writing a number as a product of factors.

$$60 = 6 \times 10.$$

In a **prime factorization**, every factor is prime.

$$60 = 6 \times 10 = (2 \times 3) \times (2 \times 5) = 2^2 \times 3 \times 5.$$

A prime cannot be split into smaller positive integer factors: its only positive divisors are 1 and itself.

Theorem (Fundamental theorem of arithmetic)

Every integer greater than 1 has a unique prime factorization, apart from the order of its factors.

Proof in the course note.

Greatest common divisors (GCD)

For integers a, b not both zero, $\gcd(a, b)$ is their **greatest positive common divisor**. That is, the largest integer g such that $g \mid a$ and $g \mid b$.

Greatest common divisors (GCD)

For integers a, b not both zero, $\gcd(a, b)$ is their **greatest positive common divisor**. That is, the largest integer g such that $g \mid a$ and $g \mid b$.

$$5 = 5 \times 1 \qquad 10 = 5 \times 2 \qquad \gcd(5, 10) = 5$$

Greatest common divisors (GCD)

For integers a, b not both zero, $\gcd(a, b)$ is their **greatest positive common divisor**. That is, the largest integer g such that $g \mid a$ and $g \mid b$.

$$5 = 5 \times 1 \qquad 10 = 5 \times 2 \qquad \gcd(5, 10) = 5$$

$$8 = 4 \times 2 \qquad 12 = 4 \times 3 \qquad \gcd(8, 12) = 4$$

Greatest common divisors (GCD)

For integers a, b not both zero, $\gcd(a, b)$ is their **greatest positive common divisor**. That is, the largest integer g such that $g \mid a$ and $g \mid b$.

$$5 = 5 \times 1 \qquad 10 = 5 \times 2 \qquad \gcd(5, 10) = 5$$

$$8 = 4 \times 2 \qquad 12 = 4 \times 3 \qquad \gcd(8, 12) = 4$$

$$60 = 12 \times 5 \qquad 84 = 12 \times 7 \qquad \gcd(60, 84) = 12$$

Coprime integers

For integers a, b, if $\gcd(a, b) = 1$, we say a and b are **coprime**.

Coprime integers

For integers a, b, if $\gcd(a, b) = 1$, we say a and b are **coprime**.

$$\gcd(8, 9) = 1,$$

$$\gcd(17, 19) = 1,$$

$$\gcd(9, 10) = 1.$$

Computing the GCD

In general, how should we compute

$$\gcd(a, b) = ?$$

Computing the GCD

In general, how should we compute

$$\gcd(a, b) = ?$$

Learning objectives

Solution 1: By prime factorization.

Solution 2: By the Euclidean algorithm.

Finding the GCD by prime factorization

If we can find the prime factorizations of a and b, we can easily find their GCD.

Finding the GCD by prime factorization

If we can find the prime factorizations of a and b, we can easily find their GCD.

$$72 = 2^3 \times 3^2 \times 5^0,$$
$$60 = 2^2 \times 3^1 \times 5^1,$$

Finding the GCD by prime factorization

If we can find the prime factorizations of a and b, we can easily find their GCD.

$$72 = 2^3 \times 3^2 \times 5^0,$$
$$60 = 2^2 \times 3^1 \times 5^1,$$
$$\gcd(72, 60) = 2^{\min(3,2)} \times 3^{\min(2,1)} \times 5^{\min(0,1)}$$

Finding the GCD by prime factorization

If we can find the prime factorizations of a and b, we can easily find their GCD.

$$72 = 2^3 \times 3^2 \times 5^0,$$
$$60 = 2^2 \times 3^1 \times 5^1,$$
$$\begin{aligned} \gcd(72, 60) &= 2^{\min(3,2)} \times 3^{\min(2,1)} \times 5^{\min(0,1)} \\ &= 2^2 \times 3^1 \times 5^0 = 12. \end{aligned}$$

Euclidean algorithm: 1. A single step

MUST UNDERSTAND Euclidean reduction

For $a \geq 0$ and $b > 0$,

$$\gcd(a, b) = \gcd(a \bmod b, b).$$

Euclidean algorithm: 1. Why the step works

Denote $r = a \bmod b$. By definition, we can write $a = bq + r$ for some integer quotient q.

To prove $\gcd(a, b) = \gcd(r, b)$, we show that, for every positive integer d,

$$(d \mid a \text{ and } d \mid b) \iff (d \mid r \text{ and } d \mid b).$$

Euclidean algorithm: 1. Why the step works

Denote $r = a \bmod b$. By definition, we can write $a = bq + r$ for some integer quotient q.
To prove $\gcd(a, b) = \gcd(r, b)$, we show that, for every positive integer d,

$$(d \mid a \text{ and } d \mid b) \quad \Longleftrightarrow \quad (d \mid r \text{ and } d \mid b).$$

From (a, b) to (r, b)

If $d \mid a$ and $d \mid b$, then

$$a \equiv 0 \pmod{d}, \quad b \equiv 0 \pmod{d}.$$

Euclidean algorithm: 1. Why the step works

Denote $r = a \bmod b$. By definition, we can write $a = bq + r$ for some integer quotient q.
To prove $\gcd(a, b) = \gcd(r, b)$, we show that, for every positive integer d,

$$(d \mid a \text{ and } d \mid b) \quad \Longleftrightarrow \quad (d \mid r \text{ and } d \mid b).$$

From (a, b) to (r, b)

If $d \mid a$ and $d \mid b$, then

$$a \equiv 0 \pmod{d}, \quad b \equiv 0 \pmod{d}.$$

Multiplication: for any integer q,

$$qb \equiv 0 \pmod{d}.$$

Euclidean algorithm: 1. Why the step works

Denote $r = a \bmod b$. By definition, we can write $a = bq + r$ for some integer quotient q.
To prove $\gcd(a, b) = \gcd(r, b)$, we show that, for every positive integer d,

$$(d \mid a \text{ and } d \mid b) \quad \Longleftrightarrow \quad (d \mid r \text{ and } d \mid b).$$

From (a, b) to (r, b)

If $d \mid a$ and $d \mid b$, then

$$a \equiv 0 \pmod{d}, \quad b \equiv 0 \pmod{d}.$$

Multiplication: for any integer q,

$$qb \equiv 0 \pmod{d}.$$

Subtraction: $r = a - qb \equiv 0 \pmod{d}$.

Euclidean algorithm: 1. Why the step works

Denote $r = a \bmod b$. By definition, we can write $a = bq + r$ for some integer quotient q.
To prove $\gcd(a, b) = \gcd(r, b)$, we show that, for every positive integer d,

$$(d \mid a \text{ and } d \mid b) \quad \Longleftrightarrow \quad (d \mid r \text{ and } d \mid b).$$

From (a, b) to (r, b)

If $d \mid a$ and $d \mid b$, then

$$a \equiv 0 \pmod{d}, \quad b \equiv 0 \pmod{d}.$$

Multiplication: for any integer q,

$$qb \equiv 0 \pmod{d}.$$

Subtraction: $r = a - qb \equiv 0 \pmod{d}$.

From (r, b) to (a, b)

Basically the same analysis.

Euclidean algorithm: 2. Repeat the step

Find $\gcd(43, 17)$ without factoring.

$$\gcd(43, 17)$$

Euclidean algorithm: 2. Repeat the step

Find $\gcd(43, 17)$ without factoring.

$$\gcd(43, 17) = \gcd(43 \bmod 17, 17) = \gcd(9, 17)$$

Euclidean algorithm: 2. Repeat the step

Find $\gcd(43, 17)$ without factoring.

$$\gcd(43, 17) = \gcd(43 \bmod 17, 17) = \gcd(9, 17)$$

$$= \gcd(9, 17 \bmod 9) = \gcd(9, 8)$$

Euclidean algorithm: 2. Repeat the step

Find $\gcd(43, 17)$ without factoring.

$$\begin{aligned}\gcd(43, 17) &= \gcd(43 \bmod 17, 17) = \gcd(9, 17) \\ &= \gcd(9, 17 \bmod 9) = \gcd(9, 8) \\ &= \gcd(9 \bmod 8, 8) = \gcd(1, 8)\end{aligned}$$

Euclidean algorithm: 2. Repeat the step

Find $\gcd(43, 17)$ without factoring.

$$\begin{aligned}\gcd(43, 17) &= \gcd(43 \bmod 17, 17) = \gcd(9, 17) \\ &= \gcd(9, 17 \bmod 9) = \gcd(9, 8) \\ &= \gcd(9 \bmod 8, 8) = \gcd(1, 8) \\ &= \gcd(1, 8 \bmod 1) = \gcd(1, 0) = \boxed{1}.\end{aligned}$$

Euclidean algorithm: 3. The full algorithm

Input: nonnegative integers a, b, not both zero.

Euclidean GCD(a, b)

repeat

Euclidean algorithm: 3. The full algorithm

Input: nonnegative integers a, b, not both zero.

Euclidean GCD(a, b)

repeat
  if $a = 0$ **or** $b = 0$:
    return the nonzero entry ▷ stop: $(a, 0)$ or $(0, b)$

Euclidean algorithm: 3. The full algorithm

Input: nonnegative integers a, b, not both zero.

Euclidean GCD(a, b)

repeat
  if $a = 0$ **or** $b = 0$:
    return the nonzero entry ▷ stop: $(a, 0)$ or $(0, b)$
  if $a \geq b$:
    $a \leftarrow a \bmod b$

Euclidean algorithm: 3. The full algorithm

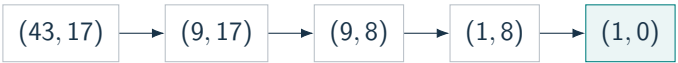
Input: nonnegative integers a, b, not both zero.

Euclidean GCD(a, b)

repeat
  if $a = 0$ **or** $b = 0$:
    return the nonzero entry ▷ stop: $(a, 0)$ or $(0, b)$
  if $a \geq b$:
    $a \leftarrow a \bmod b$
  else:
    $b \leftarrow b \bmod a$

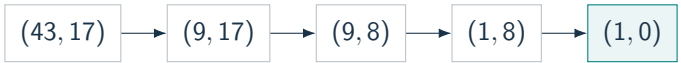
Euclidean algorithm: 4. Why it stops

Each step replaces the larger entry by a remainder smaller than the other entry.



Euclidean algorithm: 4. Why it stops

Each step replaces the larger entry by a remainder smaller than the other entry.



Exercise: Show that each update cuts the changed entry to less than half its previous value, so the algorithm takes only $O(\log \min(a, b))$ steps — very efficient!

$$43 \longrightarrow 9 \longrightarrow 1, \qquad 17 \longrightarrow 8 \longrightarrow 0.$$

Practice: Find a GCD without factoring

Compute $\gcd(84, 30)$ using the Euclidean algorithm.

Practice: Find a GCD without factoring

Compute $\gcd(84, 30)$ using the Euclidean algorithm.

$$84 = 2 \cdot 30 + 24,$$
$$30 = 1 \cdot 24 + 6,$$
$$24 = 4 \cdot 6 + 0.$$

Practice: Find a GCD without factoring

Compute $\gcd(84, 30)$ using the Euclidean algorithm.

$$84 = 2 \cdot 30 + 24,$$
$$30 = 1 \cdot 24 + 6, \qquad \gcd(84, 30) = \boxed{6}.$$
$$24 = 4 \cdot 6 + 0.$$

Contents

1. Introduction

2. Congruence

3. Modular computation and reasoning

4. GCDs and the Euclidean algorithm

5. Algorithms and recap

Number theory algorithms

Given an m-digit positive integer $N > 1$:

$$N = 13209874190283740192873401298734\ldots$$

1. **Can you tell whether N is prime?**
2. **If it is composite, can you find $p, q > 1$ with $N = p \times q$?**

Number theory algorithms: Trial division

Given an m-digit positive integer $N > 1$:

1. Can you tell whether N is prime?
2. If composite, can you find $p, q > 1$ with $N = p \times q$?

Number theory algorithms: Trial division

Given an m-digit positive integer $N > 1$:

1. Can you tell whether N is prime?
2. If composite, can you find $p, q > 1$ with $N = p \times q$?

Brute force: for $i = 2, \ldots, \lfloor \sqrt{N} \rfloor$, check whether $i \mid N$.

If one divides N, return $p = i$, $q = N/i$; otherwise, N is prime.

Exercise: Why is searching up to $\sqrt{N}$ enough?

Number theory algorithms: Trial division

Given an m-digit positive integer $N > 1$:

1. Can you tell whether N is prime?
2. If composite, can you find $p, q > 1$ with $N = p \times q$?

Brute force: for $i = 2, \ldots, \lfloor \sqrt{N} \rfloor$, check whether $i \mid N$.

If one divides N, return $p = i$, $q = N/i$; otherwise, N is prime.

Exercise: Why is searching up to $\sqrt{N}$ enough?

Worst case: exponentially many checks in m.

$$N \approx 10^m \quad \Longrightarrow \quad \sqrt{N} \approx 10^{m/2}.$$

A 1000-digit integer can have a square root close to 10^{500}.

Number theory algorithms: Polynomial time?

Given an m-digit positive integer $N > 1$:

1. Can you tell whether N is prime in time polynomial in m?

2. If composite, can you find $p, q > 1$ with $N = p \times q$ in time polynomial in m?

Number theory algorithms: Polynomial time?

Given an m-digit positive integer $N > 1$:

1. Can you tell whether N is prime in time polynomial in m?

Yes: AKS (2002) is a polynomial-time classical algorithm.

2. If composite, can you find $p, q > 1$ with $N = p \times q$ in time polynomial in m?

Number theory algorithms: Polynomial time?

Given an m-digit positive integer $N > 1$:

1. Can you tell whether N is prime in time polynomial in m?

Yes: AKS (2002) is a polynomial-time classical algorithm.

2. If composite, can you find $p, q > 1$ with $N = p \times q$ in time polynomial in m?

No known polynomial-time classical (non-quantum) algorithm.

Number theory algorithms: Factoring and GCD

Now find the factors: $N = p \times q$, with $p, q > 1$.

Hard: no known polynomial-time classical (non-quantum) algorithm.

Number theory algorithms: Factoring and GCD

Now find the factors: $N = p \times q$, with $p, q > 1$.

Hard: no known polynomial-time classical (non-quantum) algorithm.

General Number Field Sieve (GNFS): best-known for large general integers.

Expensive search

Find suitable x, y

$x^2 \equiv y^2 \pmod{N}$

→

Fast GCD step

$g = \gcd(x - y, N)$

Success if $1 < g < N$

Number theory algorithms: Factoring and GCD

Now find the factors: $N = p \times q$, with $p, q > 1$.

Hard: no known polynomial-time classical (non-quantum) algorithm.

General Number Field Sieve (GNFS): best-known for large general integers.

Expensive search

Find suitable x, y

$x^2 \equiv y^2 \pmod{N}$

→

Fast GCD step

$g = \gcd(x - y, N)$

Success if $1 < g < N$

Example: $9^2 \equiv 2^2 \pmod{77} \implies \gcd(9 - 2, 77) = 7.$

$77 = 7 \times 11.$

Summary

1. **Congruence:** compare integers through their remainders.

$$a \equiv b \pmod{m} \iff m \mid (a - b).$$

Summary

1. **Congruence:** compare integers through their remainders.

$$a \equiv b \pmod{m} \iff m \mid (a - b).$$

2. **Modular arithmetic:** $+$, $-$, and $\times$ preserve congruence.
   Not in general: division ($\div$) or reducing an exponent modulo m.

Summary

1. **Congruence:** compare integers through their remainders.

$$a \equiv b \pmod{m} \quad \Longleftrightarrow \quad m \mid (a - b).$$

2. **Modular arithmetic:** $+$, $-$, and $\times$ preserve congruence.
   Not in general: division ($\div$) or reducing an exponent modulo m.
3. **GCDs:** reduce the inputs without changing their common divisors.

$$\gcd(a, b) = \gcd(a \bmod b, b) \qquad (a \geq 0,\ b > 0).$$